Chapter route

How can an OS control heterogeneous devices through standard interfaces while controlling latency, throughput, isolation, and errors?

Prerequisites are interrupts, DMA, memory mapping, blocks, and files.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Trace an application request through device completion.
- Distinguish blocking, nonblocking, asynchronous, and buffered I/O.
- Locate bottlenecks beyond the CPU.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Overview

Track the thread, request, and buffer ownership separately. Return, completion notification, and persistence each have their own conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three I/O timelines

A read(fd, buf,4096) can hit page cache or start a device. The thread can be running, blocked, or ready; the request queued, in flight, or complete; and the buffer owned by CPU or device. These are separate state machines.

Assume one blocking thread, cache miss, and successful DMA. At t=0 the syscall starts; at 2 µs validation/submission finishes; at 3 the thread sleeps; at 102 DMA completes; at 104 the handler records completion and wakes the thread; at 110 scheduling runs it; at 112 it returns to user mode. Device service takes about100 µs, call latency112 µs, and wakeup-to-execution wait6 µs.

Before successful data is observed, it must be CPU-readable. Ready does not mean returned. Scheduling, notification, and visibility have separate synchronization conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One interface, several contracts

A descriptor denotes an opened object and access authority, not a hardware address. Regular files support seek; pipes/sockets normally do not. Terminals can apply line discipline; ioctl performs type-specific control. A common entry point does not enable every operation.

Distinguish acceptance into a buffer, driver submission, controller completion, media persistence, and remote application processing. A write returning 4096 does not universally establish all five. Durability needs a contract supported by filesystem and device; transmission success does not prove a receiving application committed. Numerical cases here are explicit teaching models, not machine measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I/O hardware

PIO, IRQ, and DMA address command/data access, notification, and transfer. Cancellation cannot free a DMA buffer while hardware may access it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ports, MMIO, controllers, and buses

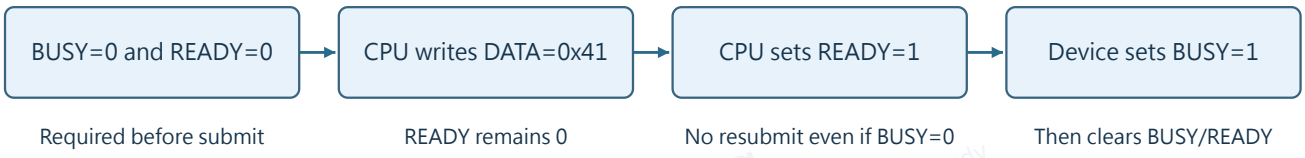
A device is the peripheral; its controller decodes commands, retains status, and transfers data. CPUs access ports through dedicated instructions or MMIO registers through protected mappings. MMIO is not ordinary RAM; caching, combining, and reordering follow architecture/driver I/O-accessor and barrier rules.

The teaching controller has STATUS, COMMAND, and DATA. STATUS bit0=BUSY, bit1=ERROR; COMMAND bit0=READY. Require BUSY=0 and READY=0, write DATA=0x41, select output, then publish READY. Publishing READY first can expose old data. Multiple CPU submitters additionally need a lock/queue; BUSY alone is not mutual exclusion.

A bus arbitrates shared transfers. Daisy-chain position/topology affects access and interrupt routing without removing permission checks.

Polling handshake and invariant

PIO output 0x41: prepare data before publishing



can_submit = !BUSY && !READY. Do not overwrite DATA before completion.

Polling handshake and invariant (continued)

Step	Actor	BUSY	READY	DATA	Next action
0	Initial	0	0	Old	Driver may submit
1	CPU writes data	0	0	0x41	Publish command
2	CPU sets READY	0	1	0x41	Controller accepts
3	Controller accepts	1	1	0x41	Output data
4	Completion clears state	0	0	0x41	CPU checks ERROR

can_submit=(BUSY==0 && READY==0). READY=1, BUSY=0 means published but not yet accepted. Even a single producer immediately attempting another byte must not overwrite this window. Testing only BUSY can replace0x41 with 0x42 before consumption.

Polling handshake and invariant (continued)

The correct trace is (0,0, old)→publish(0,1,0x41)→reject another submit, retaining0x41→accept(1,1,0x41)→complete(0,0,0x41)→publish(0,1,0x42). Under one producer, both-bit checking, and required ordering, can_submit stays false from publication through completion, preserving DATA. A device that never completes defeats termination; timeout does not prove it stopped using buffers.

Interrupts notify without moving every byte

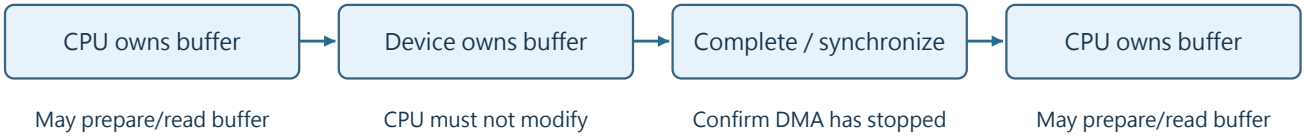
The controller raises an IRQ. At a permitted boundary the CPU saves required state and dispatches by vector. The handler checks source/status, acknowledges, updates the request, queues deferred work or wakes waiters, and returns. Shared IRQs require source identification rather than blindly completing the queue head.

Maskable interrupts, NMI, and synchronous exceptions differ; a low vector range is not uniformly NMI. Short handlers and sleepable later processing follow OS rules. Interrupts change CPU state/mode; switching threads additionally requires a scheduling context switch.

Lost wakeup occurs when a waiter reads done=false, IRQ sets true and wakes nobody, and the waiter subsequently sleeps. Under one lock, check the predicate, enqueue, and atomically release the lock while sleeping. Recheck in a while loop after waking; the IRQ publishes done under corresponding synchronization. The predicate stores completion evidence, while notification alone need not accumulate.

DMA addresses, coherence, and lifetime

4096 B receive: address mapping and ownership



Before submit: map/cache maintenance → descriptor/ownership → doorbell

After terminal: synchronize → publish → unmap/free only after hardware stops accessing

Prepare descriptor {device_addr, dma_addr, length=4096, direction=FROM_DEVICE}, mapping, and required cache maintenance before publication. Hardware transfers and notifies completion. IOMMU translation means a DMA address need not equal user VA or physical address. IOMMU confinement does not replace descriptor authorization.

DMA addresses, coherence, and lifetime (continued)

After handing the receive buffer to the device, CPU must not access it until completion and synchronization. Retain descriptor, mapping, and pages until hardware can no longer access them. Unmapping user VA does not cancel DMA. Noncoherent systems especially require the specified DMA API; a CPU barrier does not automatically invalidate caches.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transfer and notification costs

For4 KiB, PIO moving4 B per80 ns needs1024 operations, or81.92 µs CPU time. DMA submission2 µs plus IRQ/cleanup1 µs costs 3 µs CPU time, while device transfer still takes 100 µs.

A 100 ns polling check every5 µs performs about20 checks during service, costing2 µs CPU, with 0–5 µs detection delay, averaging2.5 µs under uniform phase. At2000 completions/s and1 µs/IRQ, demand is0.002 s/s=0.2% CPU. Continuous5 µs polling needs200000 checks/s, or2% CPU.

DMA still consumes memory/bus bandwidth. Overlapped CPU work does not remove system cost. Small transfers can favor PIO's lower setup cost. Compare fixed bytes, latency, and CPU denominators.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Application I/O interface

Device classes expose different operations. Short transfers, EOF, EAGAIN, and asynchronous completion require distinct branches.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Device classes, mapping, and timers

Block devices address blocks and can support random access. Character devices provide sequential data with device-specific record/seek rules. File mmap maps contents through faults/page cache; controller MMIO accesses registers.

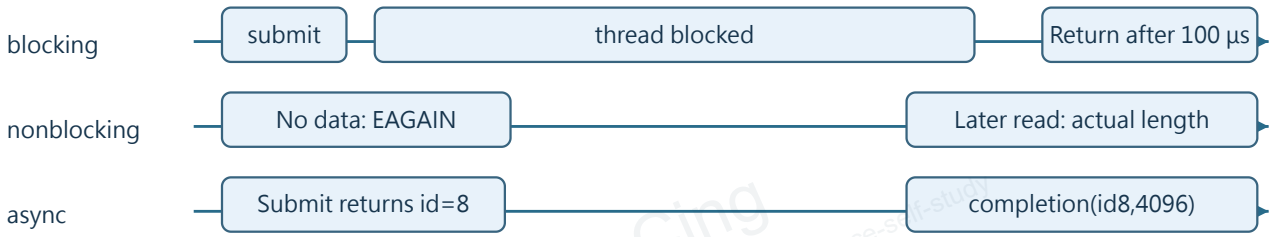
Raw/direct I/O bypasses selected services while retaining permission, alignment, lifetime, and isolation requirements. Sockets add endpoints/connections/transports. Streams do not preserve write boundaries; datagrams have message/truncation rules. Readiness transfers no bytes and does not prevent a competing thread consuming them first.

Timers provide clocks, interrupts, and deadlines. A deadline queue at now7 ms contains10,13,13. Program10, remove that expiration, program13, then remove both equal deadlines. A trigger deadline does not promise immediate callback execution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Blocking, nonblocking, and asynchronous control flow

One read, three API contracts



Device service time is unchanged; call control flow and notification timing differ.

Blocking can sleep and returns an ended operation, possibly short. Nonblocking returns EAGAIN without available data; read after poll/select still handles a competing consumer's EAGAIN. Async submission and completion differ; completion carries ID, status, actual length. Failed submission may create no request.

Blocking, nonblocking, and asynchronous control flow (continued)

A 512 B request with 200 B available can return200. Then done=200, remaining=312, dst=buf+200. Another112 gives done=312, remaining=200; subsequent zero means EOF. Deliver the short312 B or report protocol truncation. Never add -1 to done. Handle EINTR/EAGAIN under the API and progress already made.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial vector I/O

For writev lengths[100,200,300], successful250 B consumes100 then150. Retry [base1+150,50], [base2,300], totaling350 B. Reusing the original vector duplicates250 B.

Vectors reduce calls and copies into contiguous staging storage. They do not universally guarantee device zero-copy or arbitrary-length atomicity. Atomicity, order, and durability are independent. Delayed async buffers live through completion; a kernel copy changes cost/lifetime. Failed cancellation or an already-completed response does not authorize freeing memory hardware may still use.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel I/O subsystem

Scheduling, buffering, caching, spooling, errors, protection, and dispatch jointly constrain I/O.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queues, buffers, caches, and spools

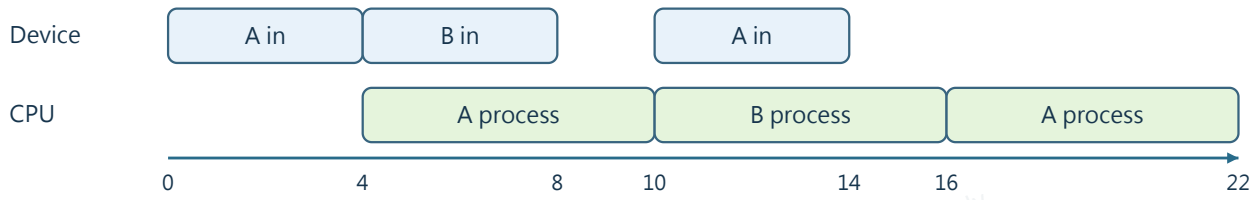
Scheduling selects service order; buffering absorbs rate/unit mismatches and preserves copy semantics; caching retains reusable data; spooling queues complete jobs for relatively exclusive devices. One region can be buffer and cache, with distinct correctness conditions.

Reads arrive for 40,10,41. Reordering10→40→41 can reduce HDD movement but cannot cross dependencies/barriers. This mechanical-seek model is not SSD latency.
Fairness/deadlines/throughput can conflict.

A dirty write-back cache can hold the only newest copy. Cache is not always an identical duplicate elsewhere. Accepted buffered write, device completion, and durability differ.

Double buffering with occupied slots

Alternate two slots; hand only a free slot to the device



ms. Input 4 ms/block; CPU 6 ms/block. B waits at 8–10; A waits at 14–16.

Fill 0–4; overlap 4–16; drain 16–22. Three blocks finish at 22 ms, not 18 ms.

Input takes 4 ms/block and CPU6 ms. One buffer takes 30 ms for three blocks. With two, A inputs0–4 and processes4–10. B inputs4–8, waits8–10, processes10–16. A inputs10–14, waits14–16, processes16–22. Total22 ms; steady period=max(4,6)=6 ms.

At4–8 the units overlap, but at 8 hardware cannot overwrite A while CPU owns it. For N blocks with full overlap and two sufficient slots,

Double buffering with occupied slots (continued)

$$T = t_{in} + t_{cpu} + (N-1)\max(t_{in}, t_{cpu}).$$

Finite batches retain fill and drain time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copy semantics, printing, and errors

An application writes ABC then changes its buffer to XYZ. If write copies/finishes consumption before returning, output remains ABC. Zero-copy async may forbid modification until completion. State the applicable contract.

The spooler persists three-page job J17 and prints. Page2 prints but acknowledgment is lost. Resending can duplicate pages; deleting can omit pages. A persistent host job ID prevents duplicate host job creation, not necessarily duplicate physical effects. Without device progress/deduplication, report outcome unknown.

Transient failures can permit retry, while non-idempotent operations cannot be blindly resent. Length/failure conventions determine valid errno use; stale errno does not make a successful call fail. Controller sense/error logs provide detail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permissions and kernel objects

P fd4→process table→open-file object{offset 8192, readable=true, ops=regular_file_ops} →inode{logical 2→physical70}→block-driver queue. Some descriptors share an open offset; independent opens need not.

Validate fd/mode/range/user pointer, then VFS dispatches by type. Page-cache hit copies directly; miss enters block/driver. Socket ops receive network bytes rather than using block 70. Name resolution and read dispatch differ; read need not walk the pathname anew.

Privilege/mapping permissions protect ports, MMIO, and control registers. Validate/appropriately pin user buffers. Device authority does not authorize arbitrary-memory DMA. Reservations can deadlock when A holds scanner waiting printer while B holds printer waiting scanner. Ordered acquisition, joint allocation, or recovery must break the cycle.

From request to hardware

Translate fd→inode→logical block→LBA→controller separately. One terminal publication does not prove exactly-once external effects.

Complete address translation

Mount /data selects device(8,2); /data/log.bin has inode 37; open creates fd4. Read4096 B at offset 8192 gives logical 2, displacement0 for 4096 B blocks. Inode maps logical 2 to device block 70. With512 B sectors, partition LBA=560, count8. Major 8/minor 2 selects driver/partition; startLBA 2048 gives whole-disk LBA 2608. The controller additionally uses a DMA mapping. Fd, inode, user VA are not LBAs. This fixed teaching model does not describe every UNIX implementation identically.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completion, timeout, and abort races

NEW→QUEUED→IN_FLIGHT→DONE or ABORTED publishes once. Timeout requests abort without proving ABORTED. Release mappings/pages only after normal completion or confirmed quiescence/abort acknowledgment.

Request8 submits0, times out100, requests abort101, completes102, and receives already-completed abort reply103. A lock/CAS protects terminal state. Completion102 wins; abort103 only releases bookkeeping without waking/unmapping twice. A caller may already have received timeout while resources remain until hardware stops.

Only a branch atomically changing nonterminal to terminal obtains publication/unmapping authority. All others see terminal and refrain. Two winners would require a forbidden second terminal transition, a contradiction. This proves at-most-once completion, neither eventual response nor exactly-once external effects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Buffer and control-block lifetimes

Stopped DMA permits buffer synchronization/unmapping, but timers/IRQs/abort callbacks may still read request state. The winner cannot immediately free its control block. Every queued/active callback already holds a stable reference and releases on exit. Canceling a timer cannot drop its reference without proving callback exit.

At101 owner, IRQ, abort hold count3. IRQ wins102, cleans DMA, exits3→2; owner delivers result and exits2→1; abort103 reads DONE, cleans bookkeeping, exits1→0, then free. An in-flight timer adds a reference. Acquisition itself requires lifetime protection; incrementing through a freed raw pointer is unsafe.

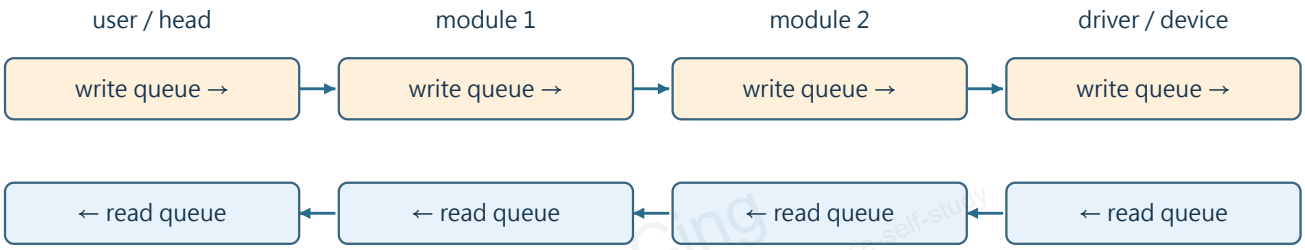
While any callback can read control state, a positive reference preserves it. Together with terminal checking this prevents double completion and use-after-free. Slot/ID reuse additionally requires generation or stable event identity against late events.

STREAMS

Each component has opposite read/write queues. High/low watermarks propagate backpressure through finite buffers.

Bidirectional message queues

Historical STREAMS: two queue directions per component



Write moves right; read moves left. Full ordinary data queues propagate backpressure upstream.

Use historical System V/Solaris STREAMS. Head meets users, driver meets devices, ioctl pushes/pops modules. Every component has read/write queues; read moves toward head, write toward driver. Data/control messages do not require every module to copy payload.

Bidirectional message queues (continued)

Put procedures can process/forward immediately or enqueue. Service procedures drain runnable queues. Read/write expose byte streams; getmsg/putmsg can preserve control/data portions. Blocking at the user head does not mean synchronous waiting through every module.

Numerical backpressure

Q capacity 1000 B, high 800, low 300, messages 200. Starting600, one accepted message gives800 and FULL, stopping upstream. Service yields600,400,200; below300 back-enables upstream. Hysteresis avoids switching repeatedly at one threshold.

Two upstream messages still consume memory. Full upstream queues eventually block the head writer. An unpausable hardware source may drop when all fill; IRQ cannot sleep indefinitely. Actual STREAMS priority/accounting is richer. This model covers ordinary fixed200 B data messages only.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Performance

Account for CPU, memory bytes, and device service per request. Little's relation needs the same boundary and finite steady-state means.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resource throughput limits

Requests4 KiB; device100000/s; CPU5 µs/request gives200000/s on one core. Memory1 GiB/s with 12 KiB traffic/request, 4 KiB DMA plus copy read 4/write 4, gives87381.33/s. Thus overall throughput≤min(100000,200000,87381.33)=87381.33/s, a bound rather than achieved performance.

Truly removing that copy gives4 KiB traffic and262144/s memory bound; device100000/s becomes limiting. Count any remaining traffic. More cores cannot overcome the original memory bound.

Batching eight arrivals10 µs apart adds70 µs for first,0 for last, average35 µs. At1 µs/IRQ, CPU cost/request falls from 1 to0.125 µs at a latency cost.

Latency and Little's relation

For finite steady-state means over one boundary, $L = \lambda W$. At80000/s and250 µs queue-plus-service response, L=20. With100 µs service, in-service count8, queue12, waiting150 µs. Queue count must not be paired with total response.

Minimum100 µs latency and100000/s throughput require average10 in flight, but queue depth10 does not guarantee that rate. Beyond saturation, more queueing can raise W without raising λ. At110000 arrivals/s and at most100000 completions/s without dropping/throttling, queue grows and finite steady-state L is unavailable.

This explicit conservation-model extension assumes neither exponential service nor the M/M/1 formula $W = 1/(\mu - \lambda)$. The latter needs additional distribution assumptions.

One read across layers

Fd4, offset 8192→inode 37 logical 2→partitionLBA 560→whole-diskLBA2608→DMA4096 B. Thread blocked, request in flight, device owns buffer. Completion establishes CPU visibility, publishes state, readies thread; scheduling later returns it.

Cache hits skip lower layers. Nonblocking can return EAGAIN without bytes; async can return submission first. Short reads, errors, permissions, and lifetime remain. Check pre/postconditions along naming/authorization→queue→device→publication→use→reclamation rather than infer durable effects from one callback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Costs and observable guarantees

PIO/DMA choose transfer, poll/IRQ detection, blocking/async call contract. These can combine. Buffer/cache/spool solve different problems; copies can improve performance while separating return from durability.

State bytes/request, CPU time/request, bus bytes/request, service time, and observation boundary. Compare equal denominators; retain finite fill/drain and steady-state stability. Exercises retain intermediate states and general arguments rather than equating a few traces with proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Silberschatz, Galvin, and Gagne, *Operating System Concepts*,9th edition, Chapter13, printed pages587–618. Cases, figures, and exercises are original. STREAMS uses the historical System V/Solaris model. Little's relation and DMA lifetime analysis are extensions under explicit assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study